

DRISHYAM

From Footage to Action

System Architecture Document

High-level Design for Real-time Operation

Document ID: DRISHYAM-ARCH-001

Version: 1.0 | 13 September 2026

Confidential – For Development Use Only

1. Architecture Goals

- Real-time detection with low latency
- Camera-agnostic ingestion (RTSP/ONVIF)
- Prefer edge inference for cost, privacy, and resilience
- Multi-tenant isolation
- Horizontal scalability of inference and backend
- Observability and auditability

2. High-Level Components

2.1 Edge Layer

Edge Node / Gateway (or AI-capable camera): Ingests RTSP streams, runs optimized detection models (YOLO-based + custom heads for weapon/violence), applies temporal filtering, generates event candidates + short clips, and forwards confirmed events + metadata to the cloud backend via secure channel. Can operate with limited connectivity (store-and-forward for critical events).

2.2 Backend Services

- API Gateway / Backend (REST + WebSocket): Authentication, authorization, camera/site/org management, alert lifecycle, user management.
- Ingestion & Event Service: Receives events from edge, persists alerts, triggers notifications.
- Notification Service: WebSocket fan-out, FCM/APNs push, optional email/SMS.
- Media Service: Stores and serves evidence clips (object storage + signed URLs).
- Optional Cloud Inference Service: Heavy models or fallback when edge is unavailable.

2.3 Data Layer

- PostgreSQL: Primary relational store (orgs, users, sites, cameras, alerts, audit logs, configs).
- Redis: Real-time presence, pub/sub for WebSocket, short-lived caches, rate limiting.
- Object Storage (S3-compatible): Evidence clips and optional thumbnails.

2.4 Frontend

- Web Dashboard (React / Next.js): Live grid, alert management, configuration, reporting.
- Mobile App (React Native or Flutter): Alert notifications, clip review, basic live view, acknowledge actions.

3. Data Flow (Happy Path)

1. Camera streams RTSP to Edge Node.
2. Edge runs inference continuously → detection + confidence + tracking.
3. Temporal filter confirms event → Edge packages metadata + short clip.
4. Edge posts event to Backend Event Service (authenticated).
5. Backend creates Alert record, stores clip reference, publishes to Redis pub/sub.
6. Notification Service pushes to connected WebSocket clients and mobile push services.
7. Operator opens alert, reviews clip, acknowledges / resolves / marks FP.
8. Status update persisted + audit log written.

4. Technology Mapping (Recommended)

Layer	Choice	Rationale
Frontend Web	Next.js (React) + TypeScript	SSR/SSG, strong ecosystem, type safety
Mobile	React Native or Flutter	Cross-platform, good push support
Backend API	NestJS (Node) or FastAPI (Python)	NestJS: structured, WS friendly; FastAPI: excellent for AI-adjacent services
AI / Edge	Python + YOLO (Ultralytics) / TensorRT / ONNX Runtime	Mature CV stack, edge optimization available

Layer	Choice	Rationale
Primary DB	PostgreSQL 15+	Relational integrity, JSONB, reliability
Cache / PubSub	Redis 7+	Speed, pub/sub, presence
Object Storage	S3 / MinIO / Cloudflare R2	Durable clip storage, signed URLs
Auth	JWT (access+refresh) + optional SSO later	Stateless, mobile-friendly
Realtime	WebSocket (Socket.IO or native)	Low-latency alert delivery
Deployment	Docker + Kubernetes / Docker Compose (early)	Reproducible, scalable

5. Deployment Topology (MVP)

- Edge nodes deployed on-site (NVIDIA Jetson / industrial PC / capable NVR) or co-located with cameras.
- Backend services in a single cloud region (or on-prem for high-security customers) behind load balancer.
- PostgreSQL managed or self-hosted with daily backups.
- Object storage in same region for low latency.
- CI/CD pipeline for backend, frontend, and edge model packages.

6. Security Architecture Highlights

- mTLS or strong token auth between Edge and Cloud
- Network isolation of edge management plane
- Least-privilege IAM for object storage
- Encryption at rest for database and clips
- Regular dependency scanning and model supply-chain checks

7. Observability

Metrics: detection latency, alert volume, false-positive rate, camera online %, API latency, error rates. Logging: structured JSON logs. Tracing: request IDs across edge → backend → notification. Alerting on pipeline health and error budgets.